

DEV_IMPL_GUIDE - SUB-WF-FRAMEWORK-01

Subscription Workflow Framework

1. Purpose

This guide explains the workflow framework as runtime flows execute.

Use it with DD_SUB-WF-FRAMEWORK-01-Subscription_Workflow_Framework-v1.0.md.

2. Runtime Actors

Runtime component	What it does
Satellite operation API	Calls framework start contract for ACTIVATE/PAUSE/etc.
subscription-workflow-api pod	Owns operation status, cancel, history, and start contract.
Framework repository	Owns subscription_operation, subscription_operation_config, and kafka_outbox.
Camunda	Runs BPMN process instances.
Worker apps	Execute BPMN external task topics.
Outbox dispatcher	Publishes operation lifecycle events.

3. Before Any Operation Can Start

Required tables:

```
subscription_operation
subscription_operation_config
kafka_outbox
```

Required indexes:

```
uq_subop_idempotency
uq_subop_in_flight_state_changing
uq_subop_in_flight_restrict
idx_subop_subscription
idx_subop_bpmn
idx_outbox_pending
```

Required config example:

```
INSERT INTO subscription_operation_config (
  operator_code,
  operation_kind,
  default_bpmn_process_key,
  operation_timeout_seconds,
  enabled,
  updated_by
)
VALUES ('WIK', 'ACTIVATE', 'sub-activate-wik', 900, TRUE, 'migration');
```

4. Flow 1 - Satellite API Starts An Operation

Example caller:

Activation API inside SUB-WF operation package

Internal command:

```
{
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_NEW",
  "operationKind": "ACTIVATE",
  "idempotencyKey": "activation:sub_01HZ_NEW:obr_01HZ_NEW_CUSTOMER",
  "requestHash": "sha256:...",
  "initiatingActorUserId": "sales-agent-wik-042",
  "initiatingActorRole": "SALES_AGENT",
  "initiatingWorkflowKind": "FUL_02_OBR_01",
  "initiatingWorkflowRef": "obr_01HZ_NEW_CUSTOMER",
  "bpmnVariables": {
    "activationTrigger": "SALES_FIELD"
  }
}
```

```
}
```

5. Flow 2 - Framework Reserves Operation In DB

The framework executes in one transaction:

1. Load subscription_operation_config.
2. Reject missing/disabled config.
3. Check idempotency.
4. Check in-flight operation.
5. Insert subscription_operation in INITIATED.
6. Insert SubscriptionOperationStarted into kafka_outbox.
7. Commit.

Insert example:

```
INSERT INTO subscription_operation (
  operation_id,
  operator_code,
  subscription_id,
  operation_kind,
  bpmn_process_key,
  initiating_actor_user_id,
  initiating_actor_role,
  initiating_workflow_kind,
  initiating_workflow_ref,
  idempotency_key,
  idempotency_request_hash,
  prior_subscription_status,
  current_state
)
VALUES (
  :operationId,
  'WIK',
  'sub_01HZ_NEW',
  'ACTIVATE',
  'sub-activate-wik',
  'sales-agent-wik-042',
  'SALES_AGENT',
  'FUL_02_OBR_01',
  'obr_01HZ_NEW_CUSTOMER',
  :idempotencyKey,
  :requestHash,
  'CREATED',
  'INITIATED'
);
```

If uq_subop_idempotency fails:

- Load existing operation.
- If hash matches, return existing operation.
- If hash differs, return 409 IDEMPOTENCY_KEY_REUSED_WITH_DIFFERENT_REQUEST.

If in-flight index fails:

- Return 409 OPERATION_IN_PROGRESS.

6. Flow 3 - Framework Starts Camunda

After DB commit, call Camunda:

```
POST /engine-rest/process-definition/key/sub-activate-wik/start HTTP/1.1
Content-Type: application/json

{
  "businessKey": "subop_01HZ_NEW_ACT",
  "variables": {
    "operationId": {"value": "subop_01HZ_NEW_ACT", "type": "String"},
    "operatorCode": {"value": "WIK", "type": "String"},
    "subscriptionId": {"value": "sub_01HZ_NEW", "type": "String"},
    "operationKind": {"value": "ACTIVATE", "type": "String"}
  }
}
```

```
}
```

Camunda response includes process instance ID. Framework updates:

```
UPDATE subscription_operation
SET bpmn_process_instance_id = :processInstanceId,
    updated_at = NOW()
WHERE operation_id = :operationId;
```

If Camunda start fails:

1. Mark operation FAILED.
2. Set failure_reason_code = BPMN_START_FAILED.
3. Insert failed event into outbox.
4. Return 502 BPMN_START_FAILED.

7. Flow 4 - Operation Status Is Read

Endpoint:

```
GET /api/subscription-operations/subop_01HZ_NEW_ACT
Authorization: Bearer <jwt>
```

Framework:

1. Authenticates caller.
2. Checks operator scope.
3. Reads subscription_operation.
4. Returns status.

Response:

```
{
  "operationId": "subop_01HZ_NEW_ACT",
  "operatorCode": "WIK",
  "subscriptionId": "sub_01HZ_NEW",
  "operationKind": "ACTIVATE",
  "currentState": "AWAITING_PAYMENT",
  "finalState": null,
  "bpmnProcessInstanceId": "pi_01HZ_NEW_ACT"
}
```

8. Flow 5 - BPMN/Worker Finalizes Operation

Worker topic:

```
subscription-operation-finalize
```

Request variables:

```
{
  "operationId": "subop_01HZ_NEW_ACT",
  "finalState": "COMPLETED"
}
```

Framework executes in one transaction:

1. Lock operation row.
2. If already terminal, complete idempotently.
3. Set current_state = finalState.
4. Set final_state.
5. Set completed_at.
6. Compute duration_ms.
7. Insert lifecycle event into outbox.
8. Commit.

9. Flow 6 - Cancel Operation

Endpoint:

```
POST /api/subscription-operations/subop_01HZ_NEW_ACT/cancel
Authorization: Bearer <bo-admin-jwt>
Content-Type: application/json

{
  "cancelReason": "CUSTOMER_CHANGED_MIND"
}
```

Framework:

1. Authenticates actor.
2. Loads operation.
3. Rejects if terminal.
4. Sends Camunda message external-cancel-requested.
5. Returns accepted response.

Camunda message:

```
POST /engine-rest/message

{
  "messageName": "external-cancel-requested",
  "correlationKeys": {
    "operationId": {"value": "subop_01HZ_NEW_ACT", "type": "String"}
  },
  "processVariables": {
    "cancelReason": {"value": "CUSTOMER_CHANGED_MIND", "type": "String"}
  }
}
```

The BPMN owns compensation and later finalizes as CANCELLED.

10. Flow 7 - Outbox Dispatcher Publishes Events

Dispatcher reads:

```
SELECT *
FROM kafka_outbox
WHERE status = 'PENDING'
AND next_attempt_at <= NOW()
ORDER BY created_at
LIMIT 100
FOR UPDATE SKIP LOCKED;
```

Then:

1. Publishes to Kafka.
2. Marks PUBLISHED.
3. On failure, increments attempts and sets next_attempt_at.
4. Marks FAILED after retry limit.

11. Flow 8 - Repair Jobs Run

Scheduled repair jobs check:

Condition	Action
INITIATED without bpmn_process_instance_id too long	Mark failed or retry Camunda start according to runbook.
In-flight operation older than timeout	Alert and inspect BPMN incident.
Outbox pending too old	Alert dispatcher.
Outbox failed rows	Alert ICN-01/ops.

12. Manual Smoke Test

1. Seed operation config.
2. Call an operation start from a satellite API.
3. Confirm operation row inserted.
4. Confirm outbox started event inserted.
5. Confirm Camunda process instance ID stored.
6. Query operation status.
7. Finalize operation through worker or test helper.
8. Confirm final state and completed event.

13. Common Mistakes

- Starting Camunda before DB reservation.
- Calling Camunda while holding subscription row lock.
- Treating every unique-key conflict as idempotency.
- Directly cancelling subscription state from cancel API.
- Publishing Kafka directly instead of outbox.